MPI Histogram of Images

Team 10	
Member	**BN**
Shams Hesham	12
Sara Alaaeldin	11
Hager Ali	37

Task Distribution

Member	Percentage	Key Deliverables	Primary Responsibilities
Member 1	**33.3%**	Dockerfile, docker-compose.yml, hostfile, run.sh	Designing the dual-subnet topology, setting up passwordless SSH, and validating cluster connectivity.
Member 2	**33.3%**	master(), worker(), MPI Tags, main() dispatch	Designing the scatter/gather protocol, implementing balanced load distribution, and managing memory buffers for MPI transfers.
Member 3	**33.3%**	compute_histogram(), save_json(), README.md	Implementing the OpenCV BGR pipeline, designing the variable bin formula, and ensuring structured JSON output.

Project Overview

This project implements a distributed image histogram computation system using MPI (Message Passing Interface) with C++ and OpenCV. The system runs across a Docker-based multi-node cluster consisting of one master node and six worker nodes, organized across two isolated networks.

The master process distributes image processing tasks to workers, each worker computes 3-channel (BGR) color histograms for its assigned images, and the master aggregates results into a structured JSON output file.

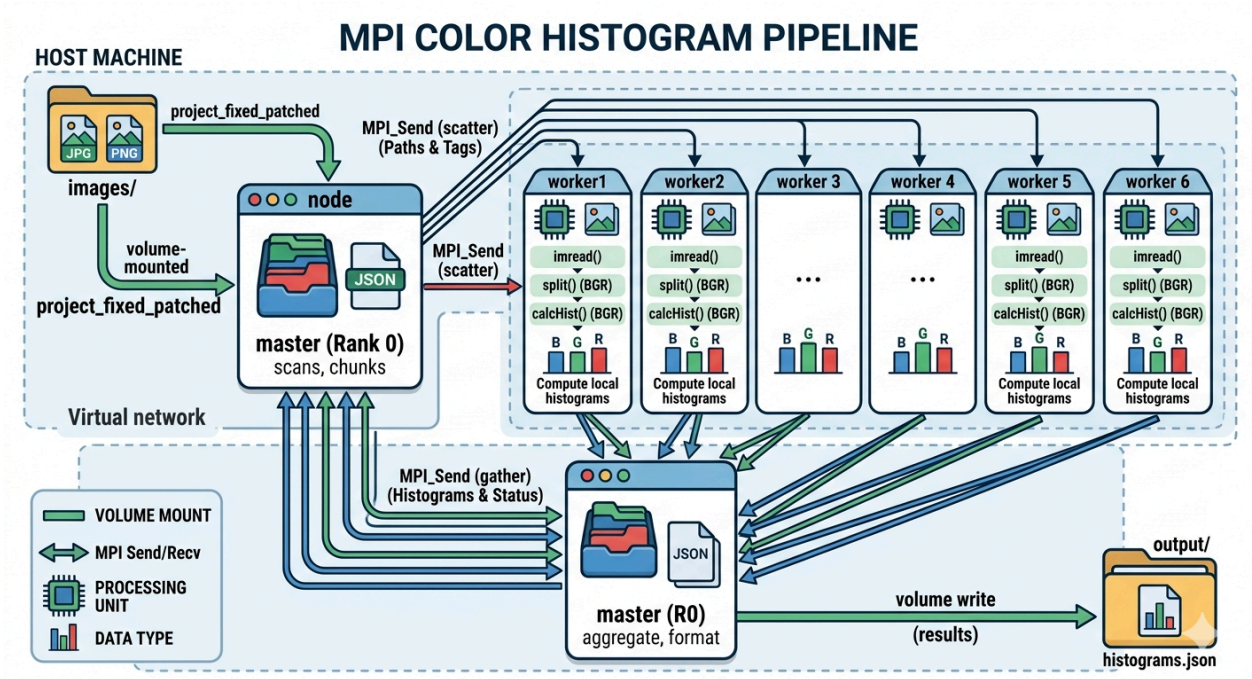
System Architecture

The cluster runs on a single Docker host machine but creates multiple isolated containers that communicate via custom networks. The master container sits on both networks, acting as a bridge, while workers are split across two separate networks to reduce congestion.

The master container performs four main responsibilities: scanning the images directory to discover all image files, distributing image paths to workers in balanced chunks, collecting computed histograms from all workers, and writing the final structured JSON output file.

Each worker container receives its assigned chunk of image paths, reads each image using OpenCV, computes separate histograms for Blue, Green, and Red channels, packages the results with metadata, and sends everything back to the master.

Shared volumes mounted on all containers provide access to input images and store the output JSON file, eliminating the need to copy files between containers.



Network Topology

Two separate bridge networks isolate worker communication paths:

mpi_network uses subnet 172.28.0.0/24 and contains the master at 172.28.0.100 plus worker1, worker2, and worker3 at addresses 172.28.0.101 through 172.28.0.103.

mpi_control_network uses subnet 172.25.0.0/24 and contains the master at 172.25.0.100 plus worker4, worker5, and worker6 at addresses 172.25.0.101 through 172.25.0.103.
This dual-network design reduces network congestion by splitting traffic across two separate network interfaces. The master maintains connectivity to all workers by being present on both networks, while workers only communicate within their assigned network.

Dockerfile

The Dockerfile creates a single container image that works for both master and worker roles. It starts from Ubuntu 22.04 and installs OpenSSH for inter-container communication, OpenMPI for distributed computing, OpenCV for image processing, and build tools for compilation.
The SSH server is configured to allow root login without password authentication, which is necessary for MPI to launch processes on remote containers. The OpenMPI hostfile is copied into the image to define the cluster topology.
The C++ source file is compiled using mpicxx with OpenCV flags, producing an executable named task2_histogram_mpi. The container runs SSH daemon in the foreground as its main process.

docker-compose.yml

This orchestration file defines seven services: one master and six workers. The master service uses privileged mode and enables IP forwarding to route traffic between the two networks. It has fixed IP addresses on both networks and depends on all workers to start first.
Workers are defined using YAML anchors to avoid repetition. Workers 1 through 3 are attached only to mpi_network, while workers 4 through 6 are attached only to mpi_control_network. All workers have the NET_ADMIN capability, allowing them to perform network administration tasks needed for MPI communication.
The two networks are defined as bridge drivers with custom IP address management configurations specifying their respective subnets. Shared volumes mount the local images directory to /app/images and the local output directory to /app/output on every container.

run.sh

This bash script manages the entire cluster lifecycle. It begins by creating the required images and output directories if they don't exist.
The script presents an interactive menu listing available Docker images and gives the user the choice to select an existing image or build a new one. When building a new image, it runs docker compose build with the no-cache flag to ensure a fresh build.
After exporting the selected image name as an environment variable, the script stops any existing containers with docker compose down and starts fresh containers in detached mode with docker compose up -d.

A five-second sleep allows SSH services to fully initialize inside all containers. The script then verifies connectivity by having the master container attempt to SSH into each worker and execute a simple echo command, reporting success or failure for each.
Finally, the script prints usage instructions showing the exact mpirun command needed to execute the histogram program, with placeholders for the window size parameter.

task2_histogram_mpi.c

This C++ file contains the complete MPI distributed computing implementation. It is structured into several functional sections.
Constants and definitions establish fixed values including master rank zero, maximum path length of 512 characters, maximum supported images of 4096, three color channels, and the maximum pixel value of 256. MPI tag constants differentiate message types: count, paths, results, and histograms.
Image discovery function scans a directory, filters for supported image extensions including jpg, jpeg, png, bmp, ppm, tiff, and tif, and builds an array of full file paths.
Histogram computation function loads an image using OpenCV, calculates the number of bins based on the window size parameter, splits the image into separate B, G, and R planes, computes a histogram for each channel using OpenCV's calcHist function, and stores the results in a flat float array where values for each channel are stored contiguously.
JSON output function writes the final results to a file with proper JSON formatting. It includes the window size parameter, total images count, total successful results, and an array of result objects. Each result object contains the image filename, status indicating success or failure, the worker rank that processed the image, and a histograms object with arrays for B, G, and R channels.
Master process logic first collects all image paths from the directory. It calculates a balanced distribution by determining the base chunk size and which workers receive an extra image. The master sends each worker their chunk size followed by the actual image paths. It then waits to receive results from all workers, aggregating image names, status flags, worker identifiers, and histogram data. Finally, it calls the JSON output function and reports statistics.
Worker process logic receives its chunk size and the array of image paths. For each image, it extracts just the filename from the full path, computes the histogram, and stores the status and its rank in metadata arrays. After processing all images, it sends the chunk size, image names, metadata, and histogram data back to the master.
The main function initializes MPI, obtains the process rank and total process count, validates that at least two processes exist, and routes execution to either master or worker function based on rank.
hostfile
This simple text file defines the cluster for MPI. Each line specifies a hostname and the number of MPI processes that can run there. The master gets one slot, and each of the six workers gets one slot, totalling seven processes.

Complete Execution Pipeline

Setup Phase begins with creating the images and output directories. Users copy their image files into the images folder. The run script is made executable with chmod.

Build and Start Phase runs the run.sh script. The interactive menu appears showing existing Docker images. Selecting the build option creates a fresh image from the Dockerfile. The script then starts all seven containers, waits for SSH initialization, and verifies connectivity.

Execution Phase requires running the mpirun command as displayed by the script. The command runs inside the master container, launches seven processes across all containers, restricts network communication to the two defined subnets, and executes the histogram program with three arguments: the images directory path, the window size parameter, and the output JSON file path.

Output Phase writes histograms.json to the mounted output directory. The master prints statistics showing how many images succeeded and failed.

Histogram Bins Formula

The number of bins for each color channel is calculated by dividing 256 by the window size parameter. A window size of 1 produces 256 bins representing full pixel resolution. A window size of 8 produces 32 bins, grouping every eight consecutive pixel values together. A window size of 16 produces 16 bins. Larger window sizes produce coarser histograms with fewer bins.

Load Balancing Strategy

The master distributes images as evenly as possible across available workers. It calculates a base chunk size by integer division of total images by number of workers. The remainder from this division determines how many workers receive one extra image. This ensures that no worker gets more than one image more than any other worker, minimizing idle time waiting for stragglers.

Communication Protocol

All communication uses MPI point-to-point functions with specific tags to identify message types. The scatter phase uses tag COUNT to send the chunk size, then tag PATHS to send the actual image path strings packed contiguously.

The gather phase uses tag COUNT for the number of results, tag PATHS for the image filenames, tag RESULTS for metadata including success status and worker rank, and tag HISTS for the flat array of histogram values.

This structured approach allows the master to receive data from multiple workers in any order without confusion about which message belongs to which phase.

Output Format

The JSON output file contains a top-level object with four fields: window_size, total_images, total_results, and results array. Each element in the results array includes the image filename, status string indicating ok or error, the worker rank that processed the image, and a histograms object containing three arrays named B, G, and R with length equal to the number of bins.
The JSON is formatted with proper indentation for human readability while maintaining valid structure for programmatic consumption.

Shared Storage Design

The images and output directories are mounted as Docker volumes on all containers. This design has several advantages: images are accessible to every container without copying, the output file is written directly to the host filesystem, and there is no need for network file transfers between containers for the image data.
The downside is that all containers are reading from the same images directory simultaneously, but since images are read-only during processing, this does not cause conflicts.

Performance Considerations

Network isolation reduces congestion by splitting six workers across two separate network bridges. Master traffic must traverse both networks, but it only sends small path strings and receives aggregated histogram data.
Load balancing prevents slow workers from becoming bottlenecks. Memory management uses static buffers for path storage to avoid dynamic allocation overhead during the main processing loop.
OpenCV's calcHist function is optimized for performance, making the per-image processing time dominated by image loading rather than histogram computation.

System Requirements

Docker version 20.10 or higher and Docker Compose version 2.0 or higher are required on the host machine. Each container requires OpenMPI 4.0 or higher and OpenCV 4.0 or higher, which the Dockerfile installs automatically. The build process requires an internet connection to download Ubuntu packages.
The system has been tested with up to several thousand images, limited primarily by memory in the master container which stores all results before writing the output file. For very large image collections, the MAX_IMAGES constant in the source code may need to be increased.